Chroma

Overview > Introduction

Overview

Introduction

Copy page

Chroma is the open-source data infrastructure for AI. It comes with everything you need to get started built-in.

Chroma gives you everything you need for retrieval: store embeddings with metadata, search with dense and sparse vectors, filter by metadata, and retrieve across text, images, and more.

What Chroma Offers

Document Storage

Store documents and metadata.

Embeddings

Use any embedding model. OpenAI, Cohere, Hugging Face, sentence-transformers, and more.

Vector Search

Dense, sparse, and hybrid search. Query by similarity and combine multiple search strategies.

Full-Text & Regex Search

Keyword and regex search over your data without embeddings.

Metadata Filtering

Filter results at query time by metadata conditions.

Multi-Modal Retrieval

Index and search images, audio, and other modalities alongside text.

Chroma

Getting Started with the Chroma SDK

Create a self-hosted or cloud database and add data to it using the Chroma SDK.

Create a Chroma Cloud Database

Create a scalable, zero-ops Chroma Cloud database to store your AI data.

Example Projects

Agentic Search

Build agents that iteratively search and refine results for complex queries.

Code Search

Index codebases to power coding agents using AST-aware chunking.

Open Source

Chroma is licensed under **Apache 2.0**. Run it locally, self-host, or use **Chroma Cloud** for a managed, serverless experience.

Was this page helpful? Yes No Suggest edits

Getting Started
Next

Chroma

Enterprise

Pricing

Changelog

Chroma

Overview › Getting Started

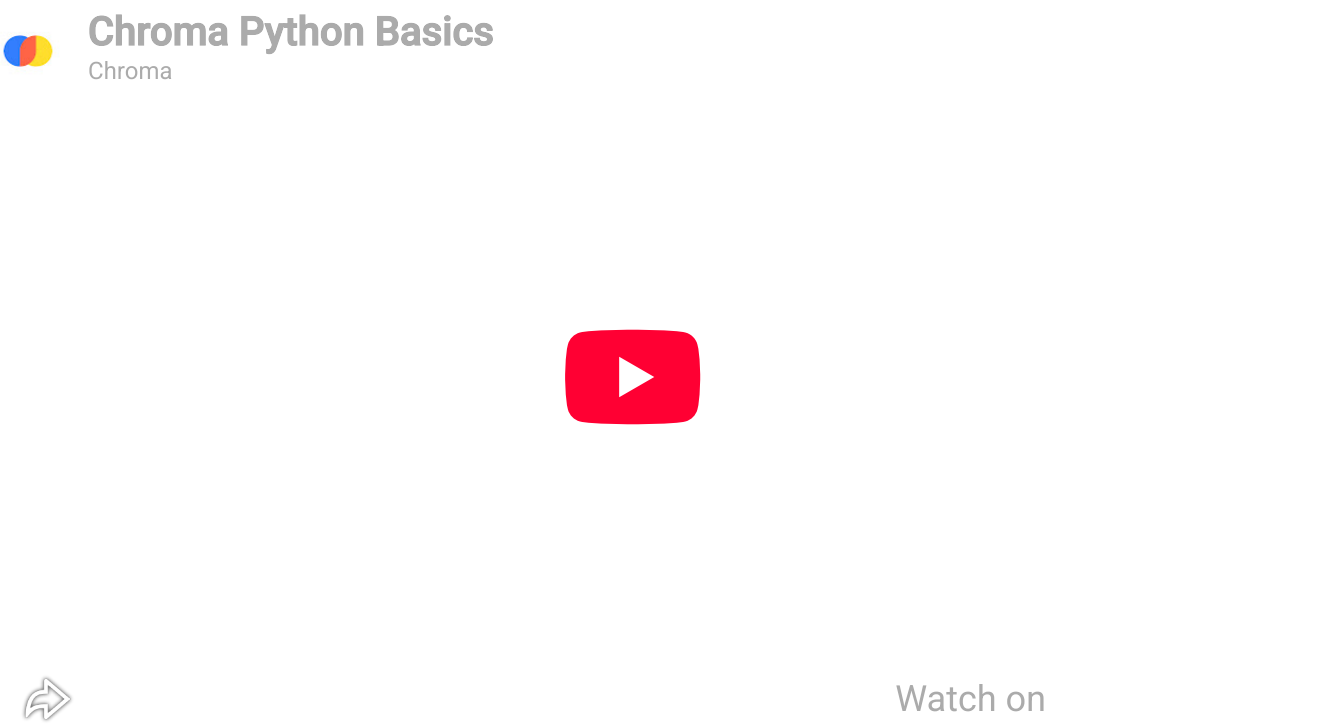
Overview

Getting Started

Copy page

Chroma is the open-source data infrastructure for AI. It comes with everything you need to get started built-in, and runs on your machine.

Python TypeScript Rust



For production, Chroma offers **Chroma Cloud** - a fast, scalable, and serverless database-as-a-service. Get started in 30 seconds - $5 in free credits included.

Install with AI

Chroma

```
In this directory create a new Python project with Chroma set up.
Use a virtual environment.

Write a small example that adds some data to a collection and queries it.
Do not delete the data from the collection when it's complete.
Run the script when you are done setting up the environment and writing the
script. The output should show what data was ingested, what was the query,
```

··· See all 28 lines

Install Manually

1 Install

pip poetry uv

```
pip install chromadb
```

2 Create a Chroma Client

Python

```
import chromadb
chroma_client = chromadb.Client()
```

3 Create a collection

Collections are where you'll store your embeddings, documents, and any additional metadata. Collections index your embeddings and documents, and enable efficient

Chroma

```
collection = chroma_client.create_collection(name="my_collection")
```

4 Add some text documents to the collection

Chroma will store your text and handle embedding and indexing automatically. You can also customize the embedding model. You must provide unique string IDs for your documents.

Python

```python
collection.add(
    ids=["id1", "id2"],
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ]
)
```

5 Query the collection

You can query the collection with a list of query texts, and Chroma will return the n most similar results. It's that easy!

Python

```python
results = collection.query(
    query_texts=["This is a query document about hawaii"], # Chroma will
    n_results=2 # how many results to return
)
print(results)
```

Chroma

From the above - you can see that our query about hawaii is semantically most similar to the document about pineapple.

Python

```
{
  'documents': [[
      'This is a document about pineapple',
      'This is a document about oranges'
  ]],
  'ids': [['id1', 'id2']],
  'distances': [[1.0404009819030762, 1.2430804967880025]],
  'uris': None,
  'data': None,
  'metadatas': [[None, None]],
  'embeddings': None,
}
```

7 Try it out yourself

What if we tried querying with “This is a document about florida”? Here is a full example.

Chroma

```
# switch \`create_collection\` to \`get_or_create_collection\` to avoid c
collection = chroma_client.get_or_create_collection(name="my_collection")

# switch \`add\` to \`upsert\` to avoid adding the same documents every t
```

See all 21 lines

Next steps

In this guide we used Chroma's **in-memory client** for simplicity. It starts a Chroma server in-memory, so any data you ingest will be lost when your program terminates. You can use the **persistent client** or run Chroma in **client-server mode** if you need data persistence.

- Learn how to **Deploy Chroma** to a server
- Join Chroma's **Discord Community** to ask questions and get help
- Follow Chroma on **X (@trychroma)** for updates

Was this page helpful? Yes No Suggest edits

Introduction
Previous

Chroma Clients
Next

Chroma

Enterprise

Pricing

Changelog

Chroma

Collections › Manage Collections

Collections

Manage Collections

Copy page

Learn how to create, get, modify, and delete Chroma collections.

Chroma lets you manage collections of embeddings, using the **collection** primitive. Collections are the fundamental unit of storage and querying in Chroma.

Creating Collections

Chroma collections are created with a name. Collection names are used in the url, so there are a few restrictions on them:

- The length of the name must be between 3 and 512 characters.
- The name must start and end with a lowercase letter or a digit, and it can contain dots, dashes, and underscores in between.
- The name must not contain two consecutive dots.
- The name must not be a valid IP address.

Python TypeScript Rust

```python
collection = client.create_collection(name="my_collection")
```

Note that collection names must be **unique** inside a Chroma database. If you try to create a collection with a name of an existing one, you will see an exception.

Embedding Functions

Chroma

Chroma also offers various embedding function, which you can provide upon creating a collection. For example, you can create a collection using the `OpenAIEmbeddingFunction`:

Python TypeScript Rust

Install the `openai` package:

pip poetry uv

```
pip install openai
```

Create your collection with the `OpenAIEmbeddingFunction`:

```
import os
from chromadb.utils.embedding_functions import OpenAIEmbeddingFunction

collection = client.create_collection(
    name="my_collection",
    embedding_function=OpenAIEmbeddingFunction(
        api_key=os.getenv("OPENAI_API_KEY"),
        model_name="text-embedding-3-small"
    )
)
```

Instead of having Chroma embed documents, you can also provide embeddings directly when **adding data** to a collection. In this case, your collection will not have an embedding function set, and you will be responsible for providing embeddings directly when adding data and querying.

Chroma

Collection Metadata

When creating collections, you can pass the optional `metadata` argument to add a mapping of metadata key-value pairs to your collections. This can be useful for adding general information about the collection like creation time, description of the data stored in the collection, and more.

Python TypeScript Rust

```python
from datetime import datetime

collection = client.create_collection(
    name="my_collection",
    embedding_function=emb_fn,
    metadata={
        "description": "my first Chroma collection",
        "created": str(datetime.now())
    }
)
```

Getting Collections

Python TypeScript Rust

There are several ways to get a collection after it was created.

The `get_collection` function will get a collection from Chroma by name. It returns a `Collection` object with `name`, `metadata`, `configuration`, and `embedding_function`.

Chroma

it doesn't exist. You can pass to it the same arguments `create_collection` expects, and the client will ignore them if the collection already exists.

```
collection = client.get_or_create_collection(
    name="my-collection",
    metadata={"description": "..."}
)
```

The `list_collections` function returns the collections you have in your Chroma database. The collections will be ordered by creation time from oldest to newest.

```
collections = client.list_collections()
```

By default, `list_collections` returns up to 100 collections. If you have more than 100 collections, or need to get only a subset of your collections, you can use the `limit` and `offset` arguments:

```
first_collections_batch = client.list_collections(limit=100) # get the first 100
second_collections_batch = client.list_collections(limit=100, offset=100) # get t
collections_subset = client.list_collections(limit=20, offset=50) # get 20 collec
```

Current versions of Chroma store the embedding function you used to create a collection on the server, so the client can resolve it for you on subsequent "get" operations. If you are running an older version of the Chroma client or server (earlier than 1.1.13), you will need to provide the same embedding function you used to create a collection when using `get_collection`:

Chroma

Modifying Collections

After a collection is created, you can modify its name, metadata and elements of its **index configuration** with the `modify` method:

Python TypeScript

```python
collection.modify(
    name="new-name",
    metadata={"description": "new description"}
)
```

Deleting Collections

You can delete a collection by name. This action will delete a collection, all of its embeddings, and associated documents and records' metadata.

> Deleting collections is destructive and not reversible

Python TypeScript

```python
client.delete_collection(name="my-collection")
```

Convenience Methods

Collections also offer a few useful convenience methods:

Chroma

```
collection.count()
collection.peek()
```

Was this page helpful? Yes No Suggest edits

Client-Server Mode
Previous

Adding Data to Chroma Collections
Next

Chroma

Enterprise

Pricing

Changelog

Chroma

Collections › Adding Data to Chroma Collections

Collections

Adding Data to Chroma Collections

Copy page

Learn how to add data to Chroma collections.

Adding Data

Use `.add` to insert new records into a collection. Each record needs a unique string `id`.

Python TypeScript Rust

```
collection.add(
    ids=["id1", "id2", "id3"],
    documents=["lorem ipsum...", "doc2", "doc3"],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapte
)
```

You must provide either `documents`, `embeddings`, or both. `metadatas` are always optional. When only providing `documents`, Chroma will generate embeddings for you using the collection's **embedding function**.

If you've already computed embeddings, pass them alongside `documents`. Chroma will store both as-is without re-embedding the documents.

Chroma

```
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2]],
    documents=["doc1", "doc2", "doc3"],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapte
)
```

If your documents are stored elsewhere, you can add just embeddings and metadata. Use the `ids` to associate records with your external documents. This is a useful pattern if your documents are very large, such as high-resolution images or videos.

Python TypeScript Rust

```
collection.add(
    ids=["id1", "id2", "id3"],
    embeddings=[[1.1, 2.3, 3.2], [4.5, 6.9, 4.4], [1.1, 2.3, 3.2]],
    metadatas=[{"chapter": 3, "verse": 16}, {"chapter": 3, "verse": 5}, {"chapte
)
```

Metadata

Metadata values can be strings, integers, floats, or booleans. Additionally, you can store arrays of these types.

Chroma

```
    documents=[ lorem ipsum... ],
    metadatas=[{
        "chapter": 3,
        "tags": ["fiction", "adventure"],
        "scores": [1, 2, 3],
    }],
)
```

All elements in an array must be the same type, and empty arrays are not allowed. You can filter on array metadata using the `$contains` and `$not_contains` operators — see **Metadata Filtering** for details.

Behaviors

- If you add a record with an ID that already exists in the collection, it will be ignored without throwing an error. In order to overwrite data in your collection, you must **update** the data.
- If the supplied embeddings don't match the dimensionality of embeddings already in the collection, an exception will be raised.

Was this page helpful? Yes No Suggest edits

Manage Collections
‹ Previous

Update Data
Next ›

Chroma

Enterprise

Pricing

Changelog

Chroma

Querying Collections › Query and Get

Querying Collections

Query and Get

Copy page

Learn how to query and retrieve data from Chroma collections.

New Search API Available

Dense vector search, hybrid search, and more are available in the new powerful **Search API** for Chroma Cloud databases.

The Query API enables nearest-neighbor similarity search over dense embeddings. Use the Get API when you want to retrieve records without similarity ranking.

Python TypeScript Rust

Query

You can query a collection to run a similarity search using `.query`:

```
collection.query(
    query_texts=["thus spake zarathustra", "the oracle speaks"]
)
```

Chroma will use the collection's **embedding function** to embed your text queries, and use the output to run a vector similarity search against your collection.

Instead of providing `query_texts`, you can provide `query_embeddings` directly. You will be required to do so if your collection does not have an embedding function attached to it.

Chroma

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]]
)
```

By default, Chroma will return 10 results per input query. You can modify this number using the `n_results` argument:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100
)
```

The `ids` argument lets you constrain the search only to records with the IDs from the provided list:

```
collection.query(
    query_embeddings=[[11.1, 12.1, 13.1], [1.1, 2.3, 3.2]],
    n_results=100,
    ids=["id1", "id2"]
)
```

Both `query` and `get` support `where` for **metadata filtering** and `where_document` for **full-text search and regex**:

Chroma

```
    where_document={"$contains": "search string"} # query records with the search
)
```

Get

Use `.get` to retrieve records by ID and/or filters without similarity ranking:

```
collection.get(ids=["id1", "id2"]) # by IDs

collection.get(limit=100, offset=0) # with pagination
```

Results Shape

Chroma returns `.query` and `.get` results in **column-major** form (arrays per field). `.query` results are grouped per input query; `.get` results are a flat list of records.

```
    embeddings: Optional[List[List[Embeddings]]]
    documents: Optional[List[List[Document]]]
    uris: Optional[List[List[URI]]]
    metadatas: Optional[List[List[Metadata]]]
    distances: Optional[List[List[float]]]
    included: Include

class GetResult(TypedDict):
    ids: List[ID]
    embeddings: Optional[Embeddings]
    documents: Optional[List[Document]]
    uris: Optional[URIs]
    metadatas: Optional[List[Metadata]]
    included: Include
```

Here is a concrete example of what these responses look like in practice:

Chroma

```
  "documents": [["Chroma stores vectors.", "Embeddings power semantic search."]],
  "metadatas": [[
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ]],
  "distances": [[0.12, 0.21]],
  "included": ["embeddings", "documents", "metadatas", "distances"]
}
// Get result
{
  "ids": ["doc_1", "doc_7"],
  "embeddings": [[1, 2, 3, 4], [1, 2, 3, 4]],
  "documents": ["Chroma stores vectors.", "Embeddings power semantic search."],
  "metadatas": [
    {"source": "docs", "topic": "intro"},
    {"source": "blog", "topic": "search"}
  ],
  "included": ["documents", "metadatas"]
}
```

In the results from the Get operation, corresponding elements in each array belong to the same document.

Python TypeScript Rust

```
result = collection.get(include=["documents", "metadatas"])
for id, document, metadata in zip(result["ids"], result["documents"], result["me
    print(id, document, metadata)
```

Query is a batch API and returns results grouped per input. A common pattern is to iterate over each query's "batch" of results, then iterate within that batch.

Chroma

```
for id, document, metadata in zip(ids, documents, metadatas):
    print(id, document, metadata)
```

Choosing Which Data is Returned

By default, Query returns `documents`, `metadatas`, and `distances`, and Get returns `documents` and `metadatas`.

Use `include` to control what comes back. `ids` are always returned.

Python TypeScript Rust

```
collection.query(
    query_texts=["my query"],
    include=["documents", "metadatas", "embeddings"],
)

collection.get(include=["documents"])
```

Was this page helpful? Yes No Suggest edits

Configure Collections
Previous

Metadata Filtering
Next

Chroma

Enterprise

Pricing

Changelog

Chroma

Querying Collections › Metadata Filtering

Querying Collections

Metadata Filtering

Copy page

Learn how to filter query results by metadata in Chroma collections.

The `where` argument in `get` and `query` is used to filter records by their metadata. For example, in this `query` operation, Chroma will only query records that have the `page` metadata field with the value `10`:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": 10}
)
```

In order to filter on metadata, you must supply a `where` filter dictionary to the query. The dictionary must have the following structure:

Python TypeScript Rust

```
{
    "metadata_field": {
        <Operator>: <Value>
    }
}
```

Using the `$eq` operator is equivalent to using the metadata field directly in your `where` filter.

Chroma

```
}

# is equivalent to

{
    "metadata_field": {
        "$eq": "search_string"
    }
}
```

For example, here we query all records whose `page` metadata field is greater than 10:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={"page": { "$gt": 10 }}
)
```

Using Logical Operators

You can also use the logical operators `$and` and `$or` to combine multiple filters.

An `$and` operator will return results that match all the filters in the list.

Chroma

```
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we query all records whose `page` metadata field is between 5 and 10:

Python TypeScript Rust

```
collection.query(
    query_texts=["first query", "second query"],
    where={
        "$and": [
            {"page": {"$gte": 5 }},
            {"page": {"$lte": 10 }},
        ]
    }
)
```

An `$or` operator will return results that match any of the filters in the list.

Chroma

```
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        },
        {
            "metadata_field": {
                <Operator>: <Value>
            }
        }
    ]
}
```

For example, here we get all records whose `color` metadata field is `red` or `blue`:

Python TypeScript Rust

```
collection.get(
    where={
        "$or": [
            {"color": "red"},
            {"color": "blue"},
        ]
    }
)
```

Using Inclusion Operators

The following inclusion operators are supported:

- `$in` - a value is in predefined list (string, int, float, bool)
- `$nin` - a value is not in predefined list (string, int, float, bool)

Chroma

```
{
  "metadata_field": {
    "$in": ["value1", "value2", "value3"]
  }
}
```

An `$nin` operator will return results where the metadata attribute is not part of a provided list (or the attribute's key is not present):

Python TypeScript Rust

```
{
  "metadata_field": {
    "$nin": ["value1", "value2", "value3"]
  }
}
```

For example, here we get all records whose `author` metadata field is in a list of possible values:

Python TypeScript Rust

```
collection.get(
    where={
       "author": {"$in": ["Rowling", "Fitzgerald", "Herbert"]}
    }
)
```

Using Array Metadata

Chroma supports storing arrays of values in metadata fields. You can use the `$contains` and `$not_contains` operators to filter records based on whether an array field includes a

Chroma

Metadata arrays can contain strings, integers, floats, or booleans. All elements in an array must be the same type.

Python TypeScript Rust

```python
collection.add(
    ids=["m1", "m2", "m3"],
    embeddings=[[1, 0, 0], [0, 1, 0], [0, 0, 1]],
    metadatas=[
        {"genres": ["action", "comedy"], "year": 2020},
        {"genres": ["drama"], "year": 2021},
        {"genres": ["action", "thriller"], "year": 2022},
    ],
)
```

Filtering with `$contains` and `$not_contains`

Use `$contains` to check if a metadata array includes a specific scalar value, and `$not_contains` to check that it does not.

Chroma

```
    where={"genres": {"$contains": "action"}}
)

# Get all records where genres does NOT contain "action"
collection.get(
    where={"genres": {"$not_contains": "action"}}
)

# Works with integer arrays too
collection.get(
    where={"scores": {"$contains": 20}}
)

# Combine with other filters
collection.get(
    where={
        "$and": [
            {"genres": {"$contains": "action"}},
            {"year": {"$gte": 2021}},
        ]
    }
)
```

Supported Array Types

Type	Python	TypeScript	Rust
String	`["a", "b"]`	`["a", "b"]`	`MetadataValue::StringArray(...)`
Integer	`[1, 2, 3]`	`[1, 2, 3]`	`MetadataValue::IntArray(...)`
Float	`[1.5, 2.5]`	`[1.5, 2.5]`	`MetadataValue::FloatArray(...)`
Boolean	`[true, false]`	`[true, false]`	`MetadataValue::BoolArray(...)`

Constraints:

Chroma

- The `$contains` value must be a scalar that matches the array's element type.

Combining with Document Search

`.get` and `.query` can handle metadata filtering combined with **document search**:

Python TypeScript Rust

```python
collection.query(
    query_texts=["doc10", "thus spake zarathustra", ...],
    n_results=10,
    where={"metadata_field": "is_equal_to_this"},
    where_document={"$contains":"search_string"}
)
```

Was this page helpful? Yes No Suggest edits

Query and Get
‹ Previous

Full Text Search
Next ›

Chroma

Enterprise

Chroma